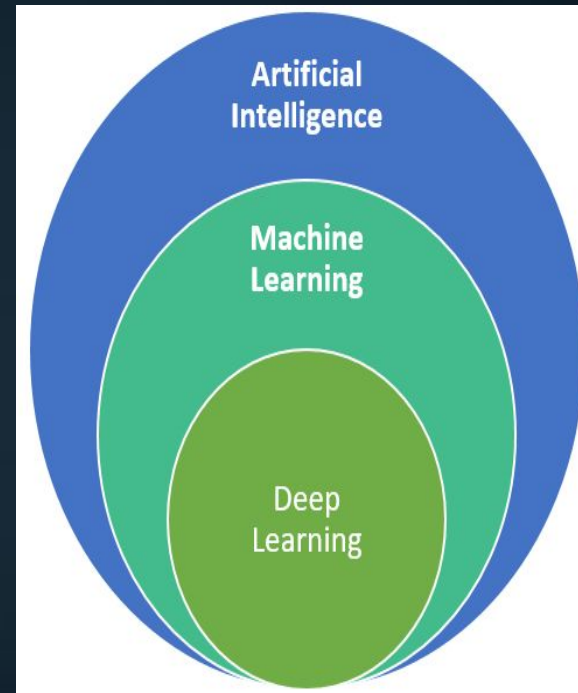
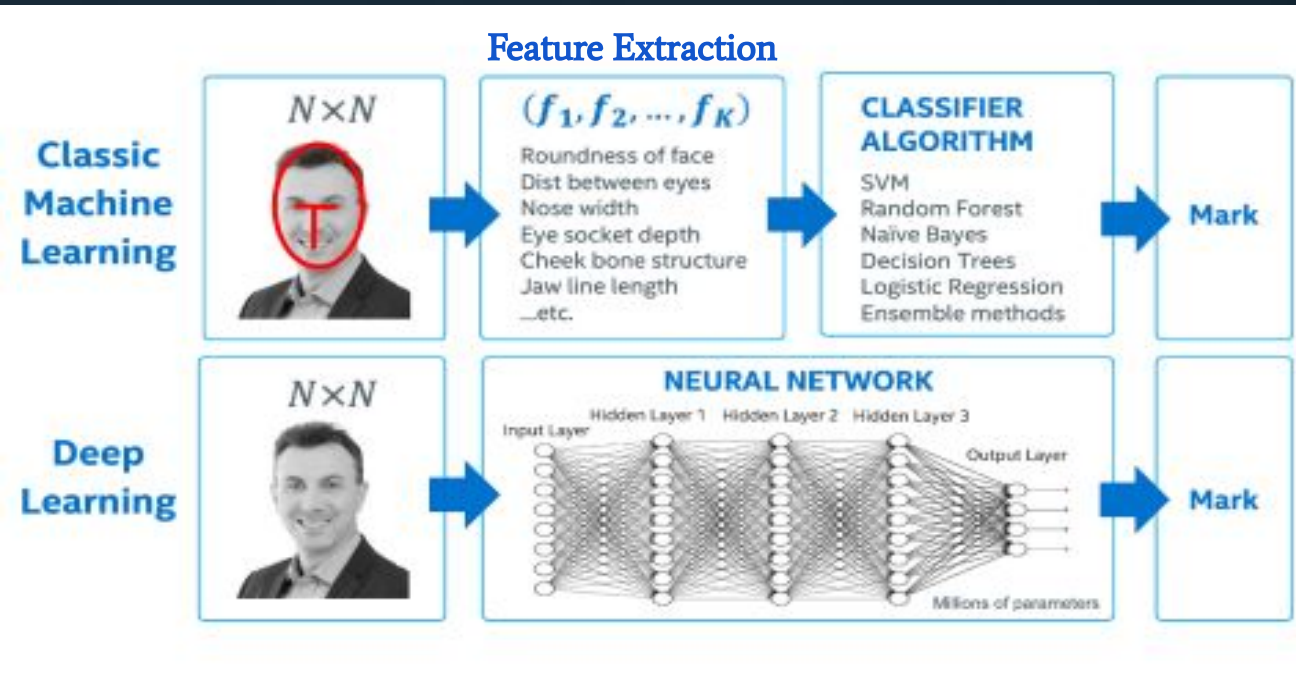


Intro to Deep Learning

Agenda

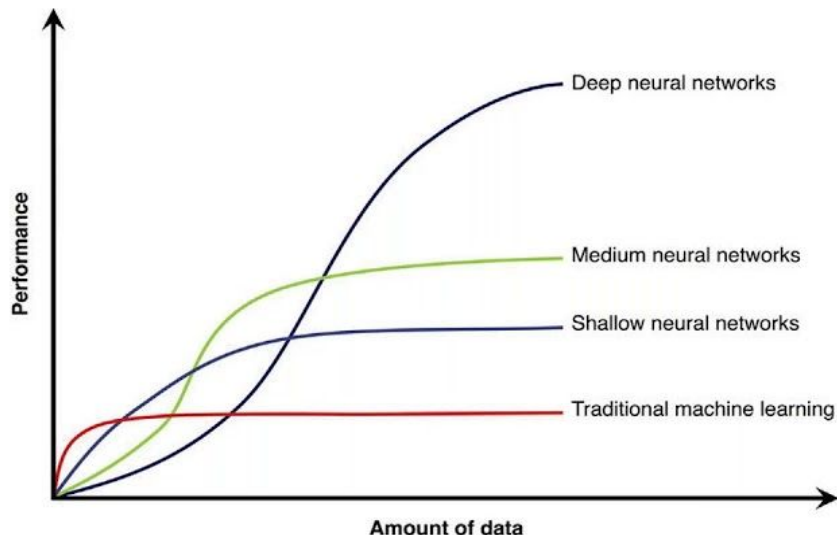
- Machine Learning vs. Deep Learning
- Why Deep Learning Now ?
- Neuron (Perceptron)
- ANN
- Activation Function
- Forward Propagation
- Backpropagation
- Different Regularization Techniques in Deep Learning

Machine Learning vs. Deep Learning



Why Deep Learning Now?

- When the amount of data is increased, machine learning techniques are insufficient in terms of performance and deep learning gives better performance like accuracy.

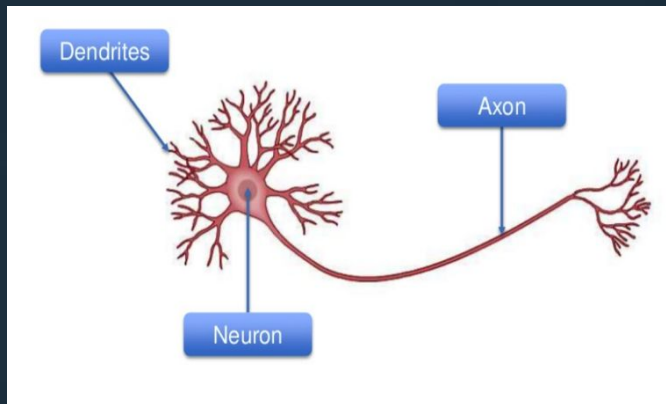


Why Now?

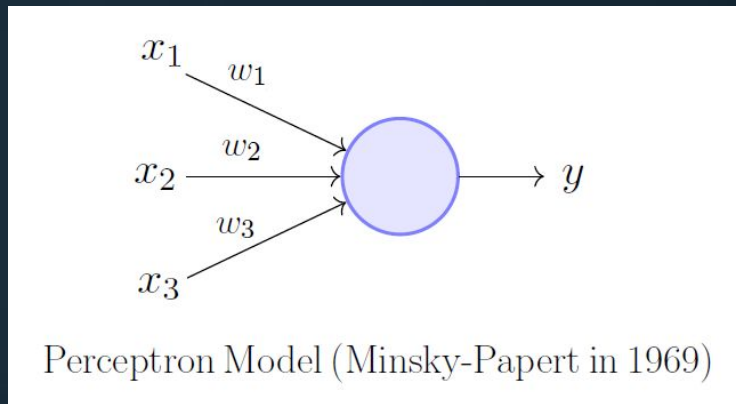
- *Algorithm Advancements*
- *GPU Computing*
- *Availability of Larger Training Data*

Foundations of DL: Neuron

- The neuron (Also, called Perceptron) is an imitation of what has been observed in the human brain.

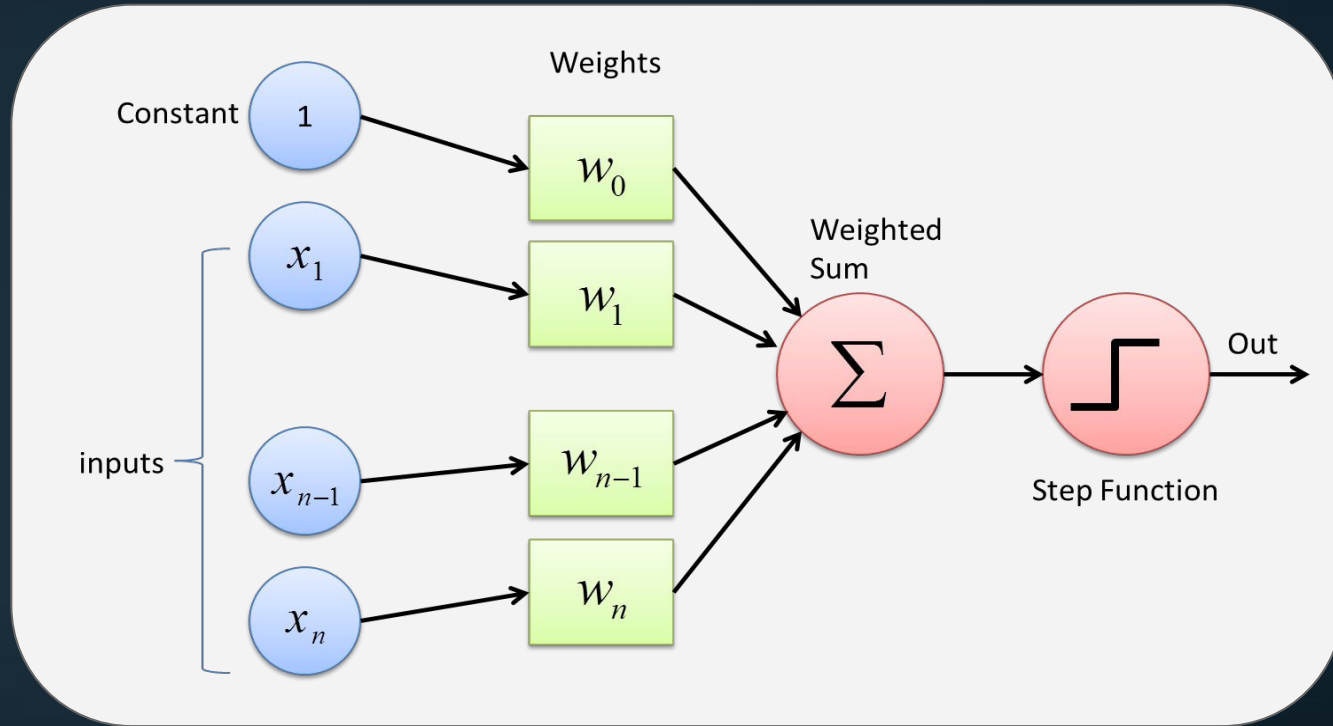


Biological Neuron



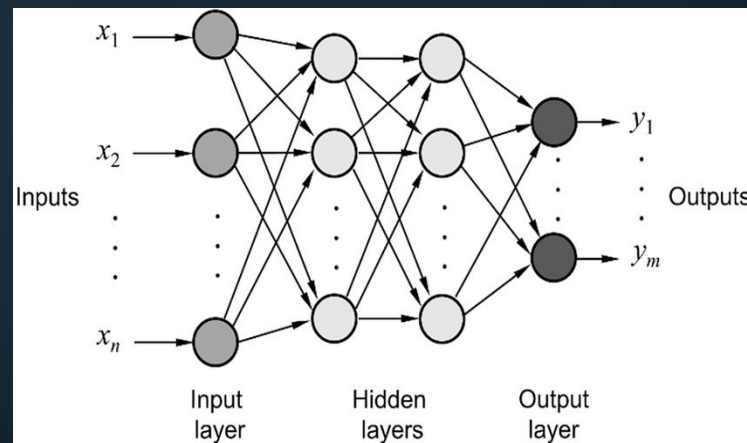
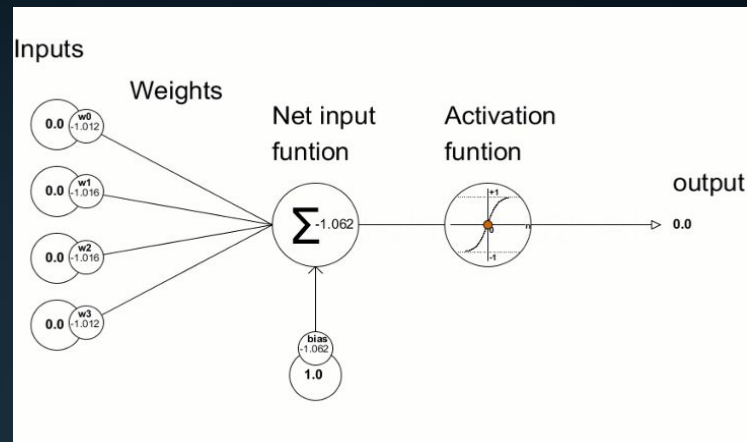
The basic unit of the neural network

Foundations of DL: Neuron



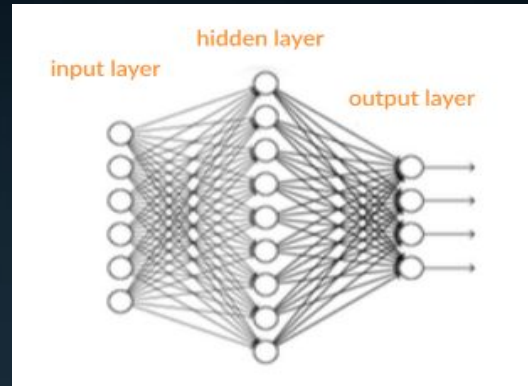
Foundations of DL: ANN

- Every neuron has a **weight**
- Based on each weight, the Neural Network decides what information is important.
- Weights are how Neural Networks learn
- The output is also controlled through Activation Function
- **Artificial Neural Network** is combinations of basic neurons arranged in multiple layers as a network.

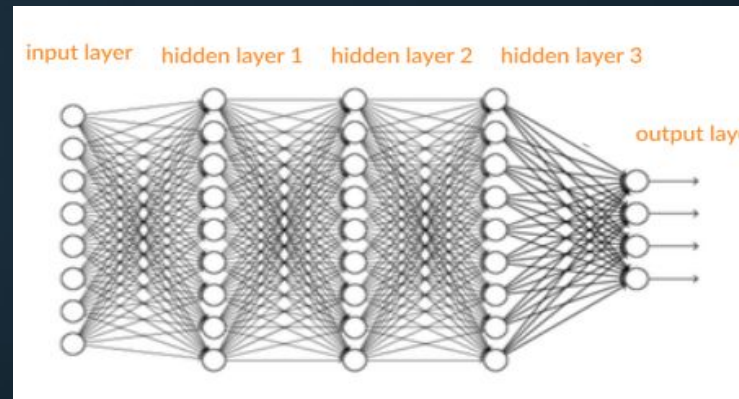


Foundations of DL: ANN

- ANN are comprised of neurons, each of which makes simple decisions.
- Together, the neurons can provide accurate answers to some complex problems.
- **Shallow NN** has only *one* “hidden layer”.
- A **Deep Neural Network** (DNN) commonly has many hidden layers of neurons.
- Neural Network Complexity increases with the number of hidden layers.

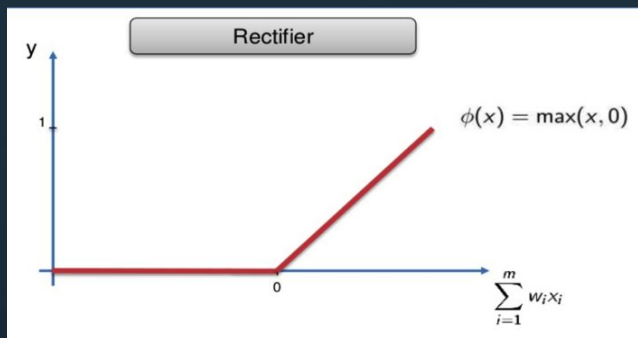
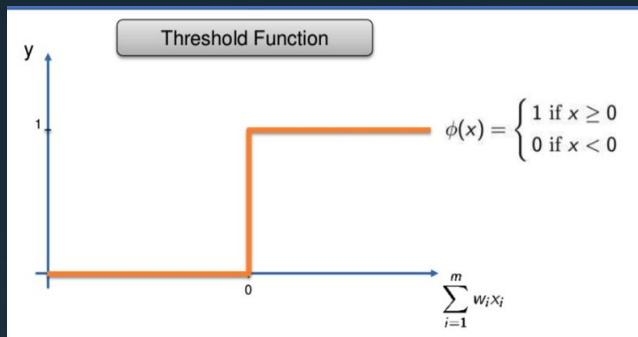


Shallow Neural Network

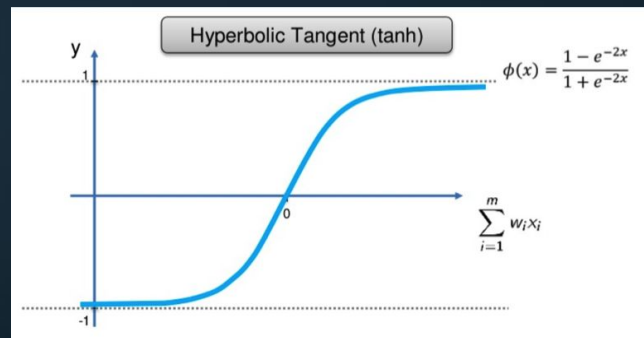
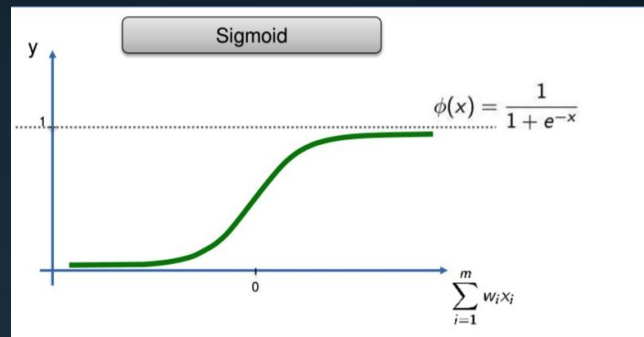
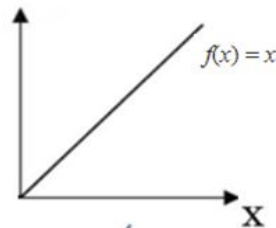


Deep Neural Network

Foundations of DL: Activation Function

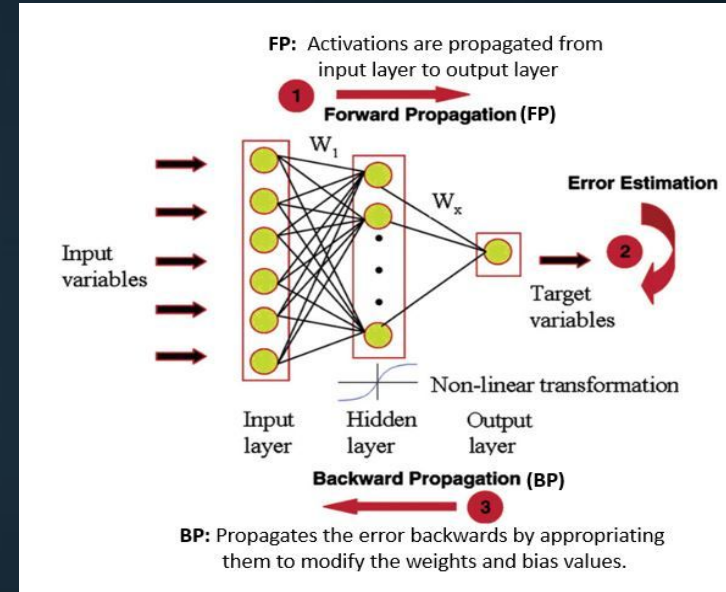


Identity Function



How do NN work (learn)?

1. **Initialization**: initial weights are applied to all the neurons.
2. **Forward propagation**: Neurons generate their outputs and pass them on to the next layer, until eventually the network generates an output.
3. **Error function**: captures the delta between the correct output and the actual output of the model.
4. **Backpropagation**: calculate the gradients to update the weights to to minimize the error.
5. Repeat steps 2-4 after each observation or after a batch of observations (Batch learning).
6. When the whole training set passes through the Neural Network, that makes an ***epoch***. Redo more epochs.



How the weights are updated?

- Weights can be updated after every sample in the training set (Stochastic), but this is usually not practical.
- Typically, a **batch** of samples is run in one big forward pass, and then backpropagation performed on the aggregate result.
- Running the **entire training set** through the neural network (Forward + Backward) is called an **epoch**.
- The batch size controls the number of training samples to work through before the model's internal parameters are updated.
- The number of epochs controls the number of complete passes through the training dataset.

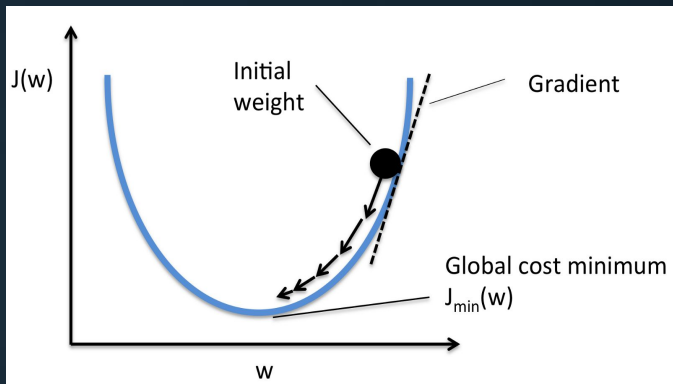
Batch vs. Epoch: Worked Example

- Assume you have a dataset with 200 samples (rows of data) and you choose a batch size of 5 and 1,000 epochs.
- This means that the dataset will be divided into 40 batches, each with five samples. The model weights will be updated after each batch of five samples.
- This also means that one epoch will involve 40 batches or 40 updates to the model.
- With 1,000 epochs, the model will be exposed to or pass through the whole dataset 1,000 times. That is a total of 40,000 batches during the entire training process.
 - Batch Gradient Descent : Batch Size = Size of Training Set
 - Stochastic Gradient Descent : Batch Size = 1
 - Mini-Batch Gradient Descent : $1 < \text{Batch Size} < \text{Size of Training Set}$



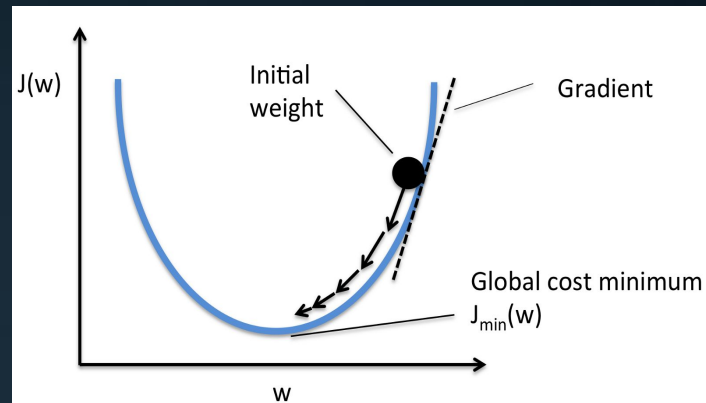
How Backpropagation Works?

- Backpropagation is simply an algorithm which performs a highly efficient search for the optimal weight values, using the ***gradient descent***.
- The gradient descent force updates the weights towards global minimum of loss function. The amount of iterations needed to converge depends on the learning rate, the network meta-parameters, and the optimization method used.



How Backpropagation Works?

- Backpropagation algorithm calculates how much the output values are affected by each of the weights in the model.
- To do this, it calculates partial derivatives (**gradients**), going back from the error function to a specific neuron and its weight. This provides complete traceability from total errors, back to a specific weight which contributed to that error.
- The result of backpropagation is a set of weights that minimize the error function.



$$w = w - \alpha \frac{\partial J(w, b)}{\partial w}$$
$$b = b - \alpha \frac{\partial J(w, b)}{\partial b}$$

Practical Tips

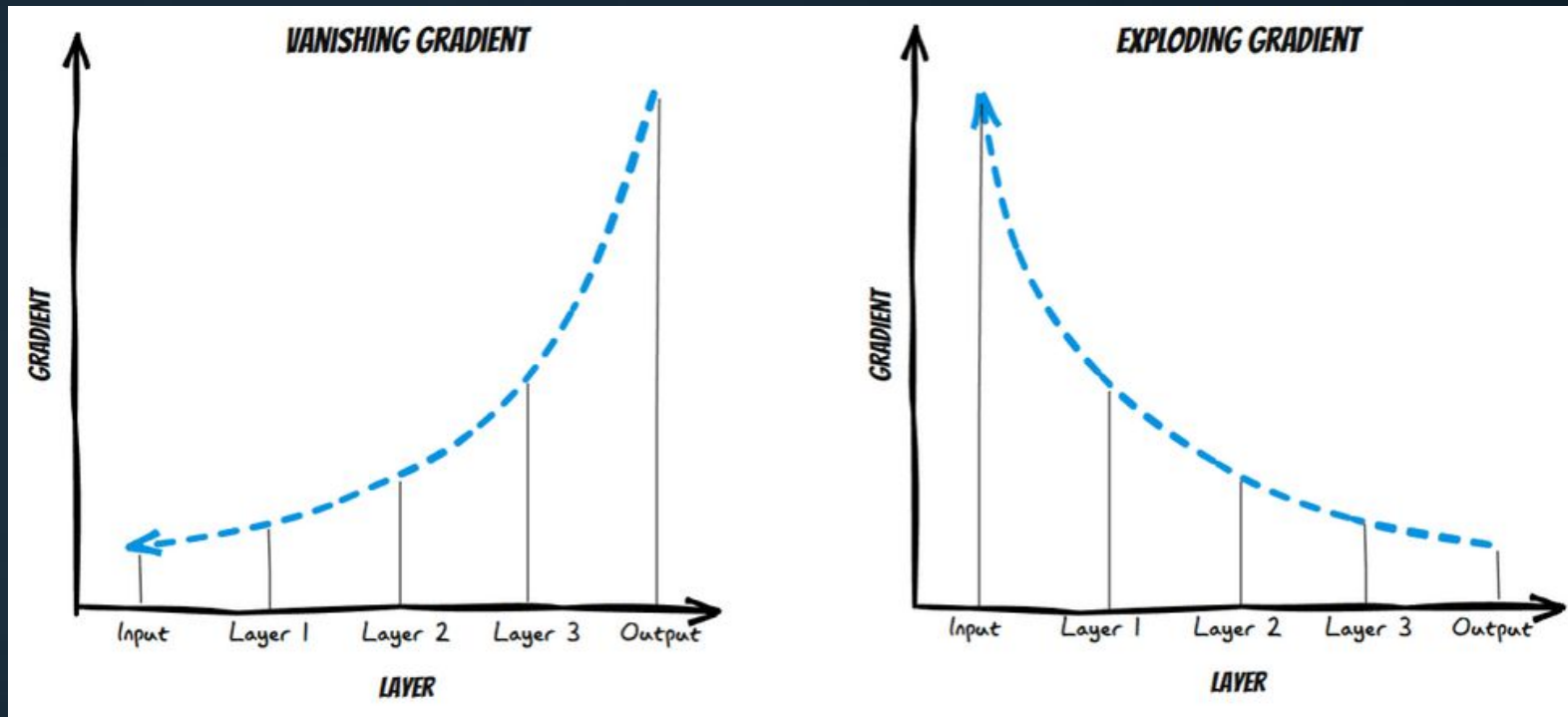
Sequential Model

```
# Build the model.  
model = Sequential([  
    Dense(64, activation='relu', input_shape=(784,)),  
    Dense(64, activation='relu'),  
    Dense(10, activation='softmax')  
])
```

- ***Sequential model*** : a single stack of layers connected sequentially
- ***Dense*** : Fully connected layer
 - ◆ every neuron in the layer is connected to every neuron in the next layer

Vanishing & Exploding Gradients

Vanishing & Exploding Gradients



Vanishing & Exploding Gradients

- ***Vanishing Gradients*** : gradients get smaller and smaller
 - Nearly Unchanged Weights
 - Hard to converge
- ***Exploding Gradients*** : gradients get larger and larger
 - Changing Weights
 - Tends to diverge
 - Famous problem with Recurrent Neural Network (RNN)
- ***Suggested Solutions:***
 - *Better Weight Initialization*
 - *Better Activation Function*

Weights Initialization

Tweaking Weight Initialization

```
tf.keras.layers.Dense(  
    units,  
    activation=None,  
    use_bias=True,  
    kernel_initializer="glorot_uniform",  
    bias_initializer="zeros",  
    kernel_regularizer=None,  
    bias_regularizer=None,  
    activity_regularizer=None,  
    kernel_constraint=None,  
    bias_constraint=None,  
    **kwargs  
)
```

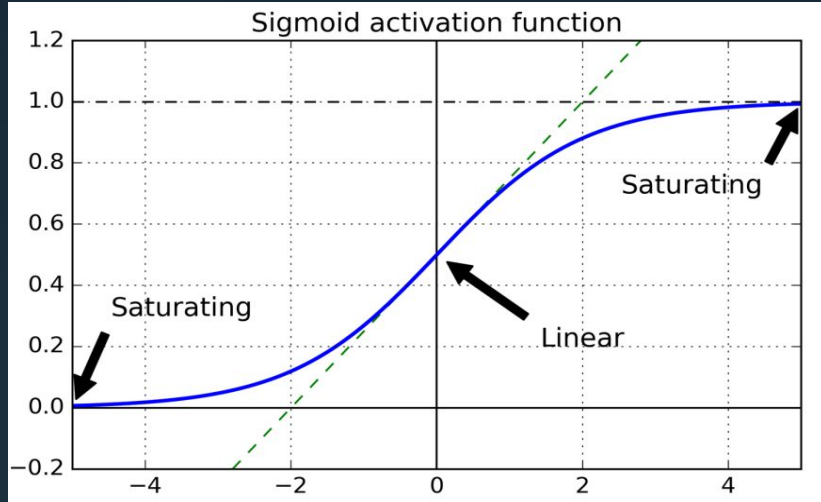
- https://keras.io/api/layers/core_layers/dense/
- <https://keras.io/api/layers/initializers/>
- https://www.tensorflow.org/api_docs/python/tf/keras/initializers

Initialization	Activation functions	σ^2 (Normal)
Glorot	None, tanh, logistic, softmax	$1 / fan_{avg}$
He	ReLU and variants	$2 / fan_{in}$
LeCun	SELU	$1 / fan_{in}$

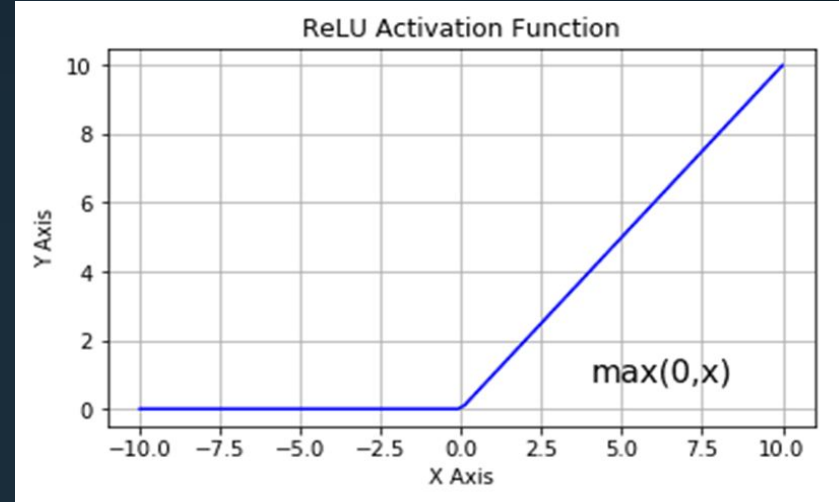
(for the uniform distribution, just compute $r = \sqrt{3\sigma^2}$)

Activation Functions

Better Activation Functions (For Hidden Layers)

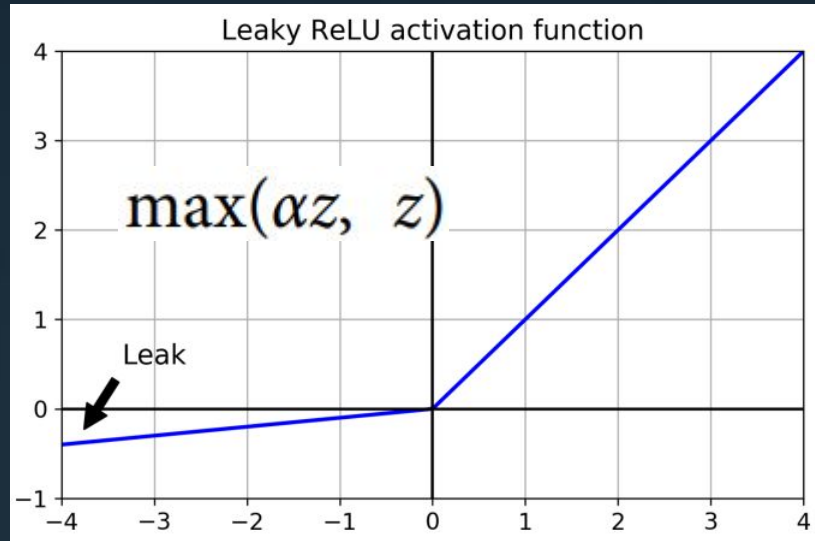


Saturating Problem



Dying Relu Problem

Relu Optimization: Leaky Relu



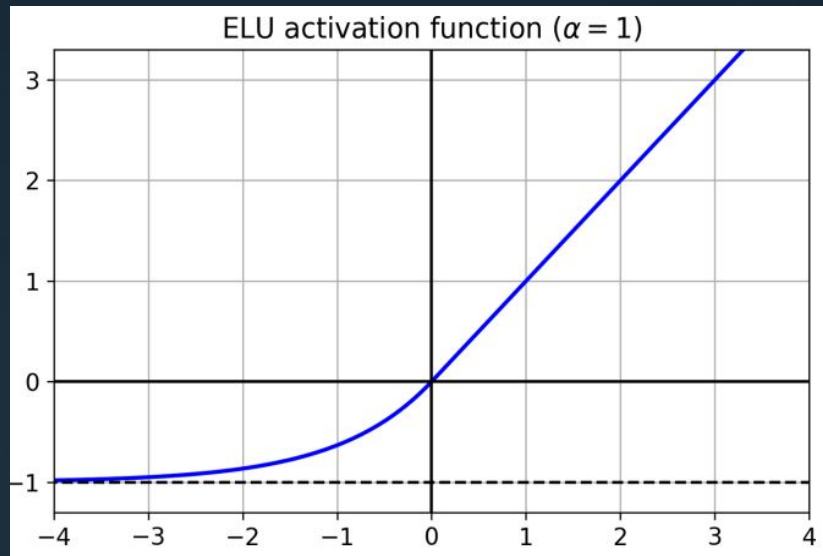
```
from tensorflow.keras.layers import LeakyReLU
```

```
model = Sequential()  
model.add(Dense(64, input_shape=(784,)))  
model.add(LeakyReLU(alpha=0.1))  
model.add(Dense(64))  
model.add(LeakyReLU(alpha=0.1))
```

- <https://keras.io/api/layers/activations/>
- https://keras.io/api/layers/activation_layers/



Relu Optimization: Exponential Linear Unit (ELU)



$$\text{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

```
from tensorflow.keras.layers import ELU

model = Sequential([
    Dense(64, input_shape=(784,)),
    ELU(alpha=1.0),
    Dense(64, activation='elu'),
    Dense(10, activation='softmax')])
```

Relu Optimization: Scaled Exponential Linear Unit (SELU)

The Scaled Exponential Linear Unit (SELU) activation function is defined as:

- if $x > 0$: return $scale * x$
- if $x < 0$: return $scale * alpha * (\exp(x) - 1)$

where $alpha$ and $scale$ are pre-defined constants ($alpha=1.67326324$ and $scale=1.05070098$).

```
model = Sequential([
    Dense(64, input_shape=(784, ), activation='selu', kernel_initializer='lecun_normal'),
    Dense(64, activation='selu', kernel_initializer='lecun_normal'),
    Dense(10, activation='softmax')])
```



Relu Optimization: Scaled Exponential Linear Unit (SELU)

```
from tensorflow.keras.layers import Activation
from tensorflow.keras.activations import elu

def selu(x, alpha=1.67326, scale=1.0507):
    return scale * elu(x, alpha)

custom_selu = lambda x: selu(x, alpha=1, scale=1)

model = Sequential([
    Dense(64, input_shape=(784, ), kernel_initializer='lecun_normal'),
    Activation(custom_selu),
    Dense(64, kernel_initializer='lecun_normal'),
    Activation(custom_selu),
    Dense(10, activation='softmax')])
```

Activation Functions

- For Hidden Layers: Relu and its derivatives are fine
- For Output layer:
 - *Regression*: one node per output, linear activation (or None)
 - *Binary Clf*: one node, Sigmoid
 - *Multi-class clf*: one node per class, softmax activation
 - *Multi-label clf*: one node per label, sigmoid activation

Classification Types

Binary Classification



- Spam
- Not spam

Multiclass Classification



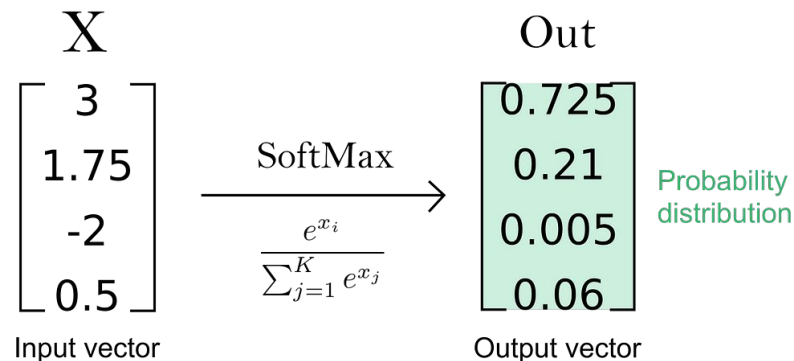
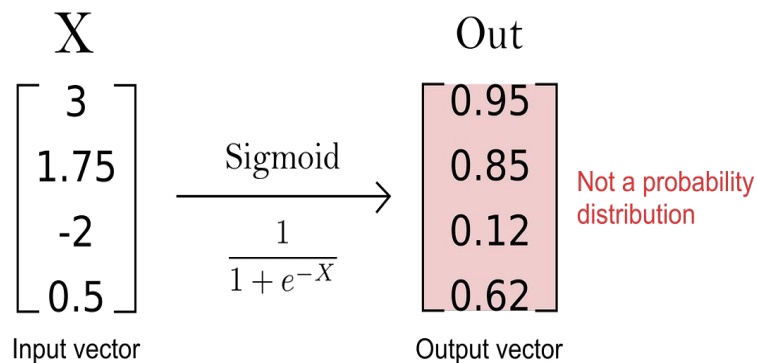
- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Multi-label Classification



- Dog
- Cat
- Horse
- Fish
- Bird
- ...

Sigmoid vs. Softmax



Binary and Multilabel Clf.

Multi-class Clf.

Batch Normalization

Batch Normalization

- A method used to make training of artificial neural networks *faster and more stable* through normalization of the layers' inputs by re-centering and re-scaling.
- The operation lets the model learn the optimal scale and mean of each of the layer's inputs.
- In many cases, if you add a BN layer as the very first layer of your neural network, you do not need to standardize your training set (e.g., using a StandardScaler); the BN layer will do it for you (well, approximately, since it only looks at one batch at a time, and it can also rescale and shift each input feature).

Batch Normalization

```
# Batch Normalization
from tensorflow.keras.layers import BatchNormalization

model = Sequential([
    BatchNormalization(),
    Dense(64, input_shape=(784, ), activation='relu'),
    BatchNormalization(),
    Dense(64, activation='relu'),
    BatchNormalization(),
    Dense(10, activation='softmax')])
```

<https://machinelearningmastery.com/batch-normalization-for-training-of-deep-neural-networks/>



Optimizers

Better Optimizers

- Training a very large deep neural network can be painfully slow.
- So far we have seen 3 ways to speed up training (and reach a better solution):
 - Applying a good initialization strategy
 - Using a good activation function
 - Using Batch Normalization
- Another huge speed boost may come from:
 - Using a faster optimizer than the regular Gradient Descent optimizer
 - Using parts of a pretrained network (Transfer Learning).

Better Optimizers: SGD with momentum

- Gradient descent is an optimization algorithm that uses the gradient of the objective function to navigate the search space.
- Gradient descent can be accelerated by using *momentum* from past updates to the search position.

- SGD

$$W_{t+1} = W_t - \alpha \cdot g$$



- SGD with momentum

$$v_{t+1} = \mu \cdot v_t - \alpha \cdot g$$

$$W_{t+1} = W_t + v_{t+1}$$

Better Optimizers : Nesterov Accelerated Gradient

- Nesterov momentum optimization, measures the gradient of the cost function not at the local position W_t but slightly ahead in the direction of the momentum

- SGD with momentum

$$v_{t+1} = \mu \cdot v_t - \alpha \cdot g$$

$$W_{t+1} = W_t + v_{t+1}$$



- Nesterov Accelerated Gradient (NAG)

$$v_{t+1} = \mu \cdot v_t - \alpha \cdot \nabla f(W_t + \mu \cdot v_t)$$

$$W_{t+1} = W_t + v_{t+1}$$

Better Optimizers : SGD with options

```
from keras.optimizers import SGD

# SGD
optimizer = SGD(learning_rate=0.01)

# SGD with momentum
optimizer = SGD(learning_rate=0.01, momentum=0.9)

# Nesterov Accelerated Gradient (NAG)
optimizer = SGD(learning_rate=0.01, momentum=0.9, nesterov=True)
```

Better Optimizers : Adagrad

- Adagrad *adapts the learning rate* for each parameter based on the cumulative sum of squared gradients.
- AdaGrad frequently performs well for simple quadratic problems, but it often stops too early when training neural networks. The learning rate gets scaled down so much that the algorithm ends up stopping entirely before reaching the global optimum.
- So you should not use it to train deep neural networks
- It may be efficient for simpler tasks such as Linear Regression.

- Adagrad

$$v_{t+1} = v_t + g^2$$

$$W_{t+1} = W_t - \frac{\alpha}{\sqrt{v_{t+1} + \epsilon}} \cdot g$$

Better Optimizers : More from Keras

- You can get fine results with:
 - Nesterov Accelerated Gradient.
 - Nadam: Adam with Nesterov momentum.

Available optimizers

- SGD
- RMSprop
- Adam
- AdamW
- Adadelta
- Adagrad
- Adamax
- Adafactor
- Nadam
- Ftrl

<https://keras.io/api/optimizers/>



DNN Training Optimization

- We have covered:
 - Weight Initialization.
 - Activation Functions.
 - Batch Normalization.
 - Different Optimizers.

```
# Build the model.  
model = Sequential([  
    Dense(64, activation='relu', input_shape=(784,)),  
    Dense(64, activation='relu'),  
    Dense(10, activation='softmax')  
])
```

```
model.compile(optimizer=optimizer, loss=loss, metrics=['accuracy'])
```

Loss Functions

Loss Function

- *Regression*: MSE or MAE
- *Binary Clf* : Binary Cross Entropy
- *Multi-class clf*: Categorical / Sparse Categorical Cross Entropy
- *Multi-label clf*: Categorical Cross Entropy

<https://keras.io/api/losses/>

- Categorical Cross Entropy

→ expect labels to be provided in a one hot representation

- Sparse Categorical Cross Entropy

→ expect labels to be provided as integers



Loss Function

```
from keras.losses import CategoricalCrossentropy, SparseCategoricalCrossentropy

y1 = [[0, 1, 0], [0, 0, 1]]
y2 = [1, 2]
y_pred = [[0.05, 0.95, 0], [0.1, 0.8, 0.1]]

cat_entropy = CategoricalCrossentropy()
sparse_cat_entropy = SparseCategoricalCrossentropy()

print(cat_entropy(y1, y_pred).numpy())
print(sparse_cat_entropy(y2, y_pred).numpy())
```

```
y3 = [[0, 1, 1], [0, 0, 1]]
y_pred = [[0.05, 0.95, 0.9], [0.1, 0.8, 0.1]]

print(cat_entropy(y3, y_pred).numpy())
# print(sparse_cat_entropy(y1, y_pred).numpy())
```



Metrics

Metrics

- **Some Regression Metrics**
 - *Mean Squared Error*
 - *Root Mean Squared Error*
 - *Mean Absolute Error*
 - *R2 Score*
- **Some Classification Metrics**
 - *Accuracy*
 - *Precision, Recall, F1 Score, AUC*

<https://keras.io/api/metrics/>

Regularization

Regularization Techniques in Deep Learning

- Regularizers allow you to *apply penalties* on layer parameters or layer activity during optimization.
- These penalties are summed into the loss function that the network optimizes.
- These layers expose 3 keyword arguments:
 - kernel_regularizer: to apply a penalty on the layer's kernel
 - bias_regularizer: to apply a penalty on the layer's bias
 - activity_regularizer: to apply a penalty on the layer's output

<https://keras.io/api/layers/regularizers>

Regularization Techniques in Deep Learning

- *L2 & L1 Regularization*

```
from keras.regularizers import l1, l2, l1_l2

model = Sequential([
    Dense(64, input_shape=(784, ), activation='relu', kernel_regularizer=l1(0.01)),
    Dense(64, activation='relu', kernel_regularizer=l2(0.01)),
    Dense(64, activation='relu', kernel_regularizer=l1_l2(0.01, 0.01)),
    Dense(10, activation='softmax')])
```

Regularization Techniques in Deep Learning

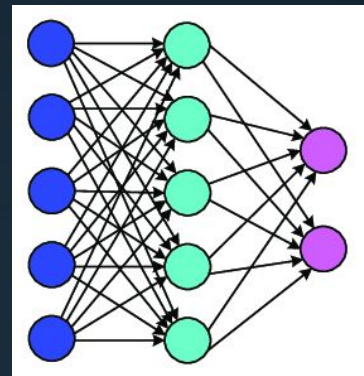
- ***Dropout***

- The most frequently technique used in deep learning.
- At every training step, every neuron (including the input neurons, but always excluding the output neurons) has a probability p of being temporarily “dropped out,” meaning it will be entirely ignored during this training step, but it may be active during the next step.
- The hyperparameter p is called the dropout rate, and it is typically set between 10% and 50%.
- ***After training, neurons don't get dropped anymore.***

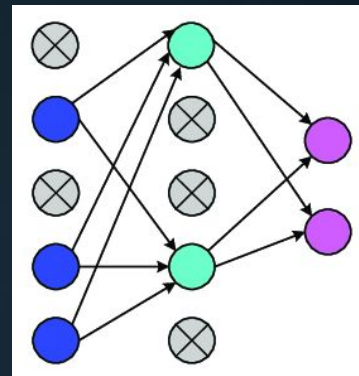
Regularization Techniques in Deep Learning

- **Dropout**

- To understand the power of dropout: realize that a unique neural network is generated at each training step. Since each neuron can be either present or absent, there are a total of 2^n possible networks (where n is the total number of droppable neurons)
- The resulting neural network can be seen as an averaging ensemble of all these smaller neural networks.



Without Dropout



With Dropout

Regularization Techniques in Deep Learning

- *Dropout*

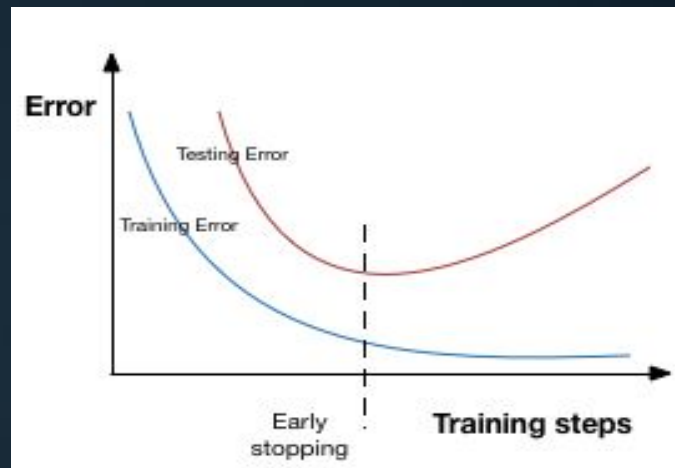
```
from keras.layers import Dropout

model = Sequential()
model.add(Dense(64, input_shape=(784, ), activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(10, activation='softmax'))
```

- If you observe that the model is overfitting, you can increase the dropout rate.
- Conversely, you should try decreasing the dropout rate if the model underfits the training set

Regularization Techniques in Deep Learning

- **Early stopping**
 - It will interrupt training when it measures no progress on the validation set for a number of epochs (defined by the patience argument), and it will optionally roll back to the best model.
 - The number of epochs can be set to a large value since training will stop automatically when there is no more progress



Regularization Techniques in Deep Learning

- *Early stopping*

```
from keras.callbacks import EarlyStopping

early_stopping = EarlyStopping(monitor='val_loss', patience=3, min_delta=0.001, restore_best_weights=True)

model.fit(X_train, y_train, epochs=100, validation_split=0.2, callbacks=[early_stopping])
```

Callbacks

Callbacks

- A callback is an object that can perform actions at various stages of training (e.g. at the start or end of an epoch, before or after a single batch, etc).
- You can use callbacks to:
 - Periodically save your model to disk
 - Do early stopping
 - and more

<https://keras.io/api/callbacks/>

Callbacks: Model Checkpoint

- *ModelCheckpoint* callback saves checkpoints of your model at regular intervals during training, by default at the end of each epoch:

```
from keras.callbacks import ModelCheckpoint, EarlyStopping

checkpoint = ModelCheckpoint('my_keras_model.h5', monitor='val_acc', save_best_only=True)
early_stopping = EarlyStopping(monitor='val_acc', patience=10, restore_best_weights=True)

model.fit(X_train, y_train, epochs=100, validation_data=(X_valid, y_valid),
          callbacks=[checkpoint, early_stopping])
```

- *Combine both callbacks to save checkpoints of your model (in case your computer crashes) and interrupt training early when there is no more progress (to avoid wasting time and resources)*



DNN Training Optimization

- We have covered:
 - Weight Initialization.
 - Activation Functions.
 - Batch Normalization.
 - Different Optimizers.
 - Different Loss Functions.
 - Different Metrics.
 - Regularization Techniques (L1, L2, Dropout, Early Stopping)
 - Callbacks (Model Checkpoint, Early Stopping)

<https://playground.tensorflow.org/>

